Smart Traffic Control (STC)

Divya Prakash Sinha 22je0331[1], Dyuti Ballav Paul 22je0335[1], Devansh Gupta 22je0308[1], Shreyas Kumar Patel 22je0925[1], and Rishit Das 22je0804[1]

[1]Team: D3SR
[1]Department of Computer Science and Engineering, IIT (ISM) Dhanbad

Abstract

Urban traffic congestion remains a critical challenge for smart city infrastructure. Traditional fixed-timing traffic signals fail to adapt to live traffic variability, leading to severe gridlocks and elevated vehicle wait times. This paper details the architecture and implementation of a real-time, adaptive traffic signal control system for a complex urban network (Patna STC, 10 signalized intersections). We propose an architecture combining a Spatial-Temporal Graph Attention Network with Proximal Policy Optimization (STGAT). The model uses 3-head attention with $K{=}3$ hop residual stacking and GRU temporal memory, producing Hold/Switch decisions at 1-second resolution per intersection. Deployed within an IoT-constrained framework, the system leverages SUMO as a simulation environment with 3D twin by Unity, MQTT (Mosquitto) for edge sensor broadcasting, and InfluxDB for time-series persistence. Furthermore, we present an ablation study comparing our GNN approach against Independent Q-Learning and Graph Diffusion Convolution. Our experiments demonstrate a significant reduction in maximum wait times compared to baseline fixed-timing approaches. This document details the architectural approach, physical sensing requirements, engineering challenges resolved, and scalable IoT deployment strategies.

Index Terms

Reinforcement Learning, Proximal Policy Optimization, Graph Attention Networks, Internet of Things, Traffic Control, Digital Twin, Edge Computing

Code Repository

I. Introduction

Modern urban areas suffer from unpredictable traffic demand, varying vehicle types with disparate passenger car unit (PCU) equivalents, and localized bottlenecks that cause cascading failures across the traffic grid. Pre-timed or actuated traffic signals do not holistically optimize the network state because they lack macroscopic spatial awareness.

With the advent of the Internet of Things (IoT) and deep reinforcement learning (DRL), traffic intersections can now act as intelligent edge devices. By communicating state arrays—such as lane-level queues and signal phases—agents can learn localized optimal switching policies. The core novelty of this paper lies in substituting isolated single-agent reinforcement learning with a *Spatial-Temporal Graph Attention Network (STGAT)*, fused with Proximal Policy Optimization (PPO). This allows a given traffic signal to weight the incoming congestion from neighboring intersections before selfishly maximizing its own throughput.

II. Problem Statement and Novelty

The objective is to minimize intersection wait times and clear dynamic queues across a network of heavily congested intersections in the Patna STC grid. The system must operate at a 1-second resolution, broadcasting metrics and applying phase-switch decisions in soft real-time.

A. What makes this challenging in Real-World Scenarios with various level of difficulties?

- **Non-Stationarity:** Traffic flow shifts dramatically due to peak hours, rendering deterministic models ineffective.
- **Heterogeneous Traffic:** Traffic in dense regions is highly mixed (motorcycles, auto-rickshaws, buses) requiring unified Passenger Car Unit (PCU) proxy metrics.
- **Ripple Effects:** A delayed green light at Node A quickly blocks the outflow of Node B. Greedy local optimization often worsens global gridlock.
- **Soft Real-Time Constraint:** The full pipeline—sensor ingestion, GNN forward pass, and PLC command dispatch—must complete within 1 second per tick.

B. Formal Problem Definition

We model the traffic network as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, A)$ where $\mathcal{V} = \{v_1, \ldots, v_N\}$ represents $N = 10$ signalized intersections, $\mathcal{E}$ represents physical road connections, and $A \in \mathbb{R}^{N \times N}$ is a weighted adjacency matrix with:

$$A_{ij} = \begin{cases} \frac{100}{d_{\text{BFS}}(v_i, v_j)+1} & \text{if } v_j \text{ reachable from } v_i \\ 100 & \text{if } i = j \text{ (self-loop)} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

where $d_{\text{BFS}}(v_i, v_j)$ is the accumulated road length from a BFS traversal that skips unsignalized nodes.

At each discrete time step t, each intersection v_i observes a local state vector:

$$\mathbf{s}_i^{(t)} = \left[\frac{q_i^{(t)}}{50}, \frac{w_i^{(t)}}{100}, \frac{\phi_i^{(t)}}{|\Phi_i|}, \min\left(\frac{\tau_i^{(t)}}{15}, 1 \right) \right] \in \mathbb{R}^4 \tag{2}$$

where q_i is the PCU-weighted queue length, w_i is the maximum accumulated wait time (seconds), ϕ_i is the current phase index, $|\Phi_i|$ is the total number of phases, and τ_i is the time elapsed since the last phase change.

The agent selects a binary action $a_i^{(t)} \in \{0, 1\}$ (Hold/Switch) for each intersection. The per-node reward is:

$$r_i^{(t)} = -\left(q_i^{(t)} + \omega \cdot w_i^{(t)} \right) - \mathbb{1}[a_i^{(t)} = 1] \cdot \lambda_{\text{switch}} \tag{3}$$

where $\omega = 0.5$ balances queue vs. wait importance, and $\lambda_{\text{switch}} = 15.0$ penalizes excessive phase cycling. The global step reward is $R^{(t)} = \sum_{i=1}^{N} r_i^{(t)}$.

C. The Novelty of STGAT in Traffic Flow

Traditional Reinforcement Learning (RL) forces agents to learn the state of the entire map as a flattened vector map, which breaks down if the map topology changes or scales up. Our STGAT approach treats the traffic grid as a localized Graph:

- **Attention Mechanism:** By using multiple attention heads, an intersection can dynamically "pay attention" to a K-hop neighbor. If Node B has an overflowing queue headed toward Node A, Node A's attention weights shift, prompting it to alter its phase to accommodate the incoming surge.
- **Shift from Independent Learning:** Independent actors behave selfishly. STGAT forces mathematical collaboration without requiring centralized brute-force computation.

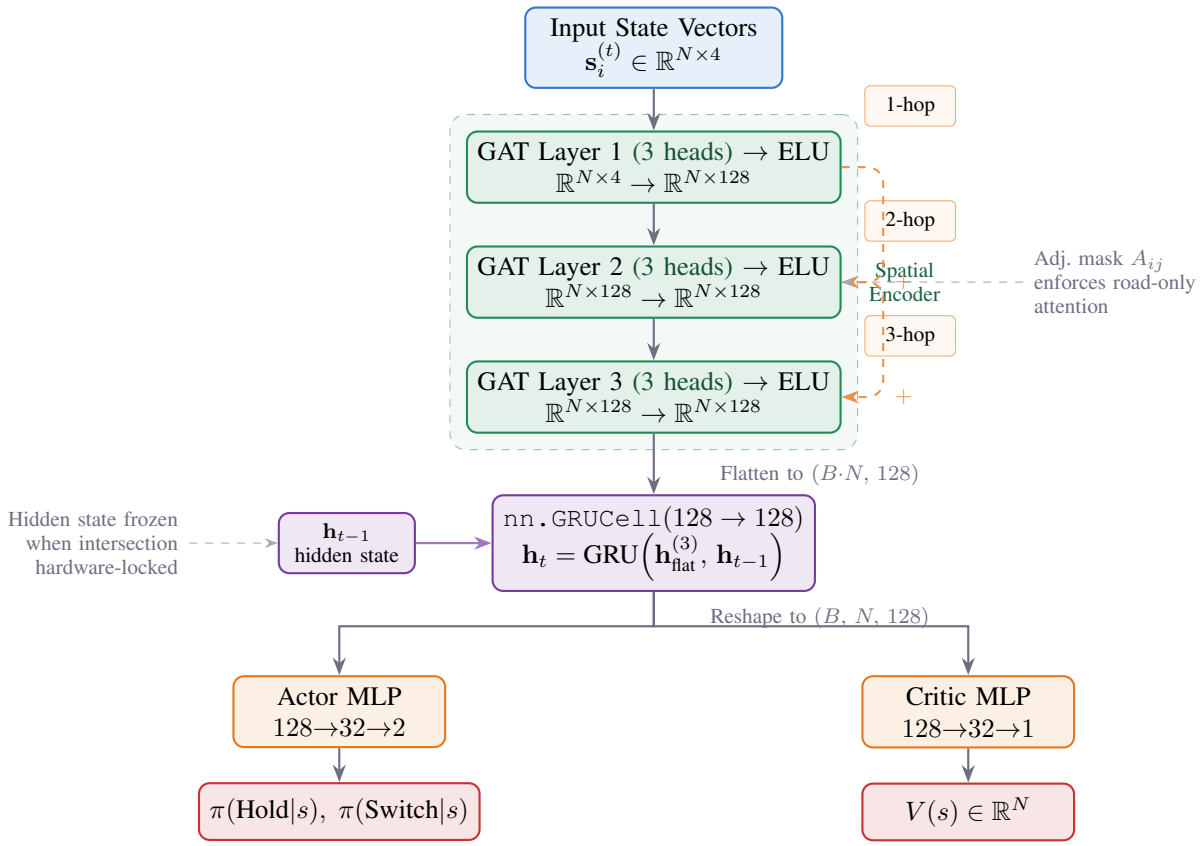

Fig. 1. STGAT model architecture: Multi-Head Graph Attention layers with K-hop residual stacking feeding into a GRU temporal memory unit, producing per-intersection Hold/Switch action probabilities through an Actor-Critic head.

III. STGAT Architecture: Detailed Walkthrough

This section provides a layer-by-layer explanation of the Spatial-Temporal Graph Attention Network as implemented in our codebase. The model (`STGAT_ActorCritic`) uses `feature_dim=4`, `hidden_dim=128`, `num_actions=2` (Hold/Switch), `k_hops=3`, and `num_heads=3`. We justify the architectural choices against alternatives such as full Transformers.

A. Why Not a Full Transformer?

A full Transformer (e.g., GPT, BERT) applies self-attention over a flat sequence of tokens, allowing every token to attend to every other token. While powerful for natural language, this design is fundamentally misaligned with traffic networks for three reasons:

1) **Topology Ignorance:** A Transformer would allow Intersection 1 to attend to Intersection 10 even if no physical road connects them. Our GAT layer masks attention using the adjacency matrix A, ensuring information flows only along real roads.
2) **Computational Overhead:** Full self-attention is $O(N^2)$ across all node pairs. Our adjacency-masked GAT is effectively $O(|E|)$ where $|E|$ is the number of edges (roads), which is significantly sparser.
3) **Redundant Mechanism:** The core innovation of the Transformer is multi-head attention — and we already incorporate it. Our architecture extracts the "secret sauce" of Transformers (multi-head attention) while constraining it to the physical graph topology, which is more principled for spatial infrastructure networks.

B. Multi-Head Graph Attention Layer (GATLayer)

Each GATLayer performs the following operations:

1) Step 1: Feature Projection into Multiple Heads: Each intersection's 4-dimensional feature vector $\mathbf{x}_i = [\text{queue}_{\text{pcu}}/50,\ \text{max_wait}/100,\ \text{phase}/N_{\text{phases}},\ \min(t_{\text{elapsed}}/15, 1)]$ is projected into $H = 3$ separate subspaces via a shared linear transformation $\mathbf{W} \in \mathbb{R}^{4\times(128\cdot 3)}$ (without bias):

$$\mathbf{W}\mathbf{h}_i = \text{Linear}(\mathbf{x}_i) \in \mathbb{R}^{H\times d_{\text{hidden}}} \tag{4}$$

Each head learns a *different perspective* of the traffic state. Intuitively:

- **Head 1** may specialize in detecting *queue pressure* from neighbors.
- **Head 2** may focus on *phase synchronization* patterns.
- **Head 3** may track *wait-time urgency* signals.

2) Step 2: Attention Scoring: For each head, the model computes pairwise attention coefficients between connected intersections using separate learnable source and destination vectors $\mathbf{a}_{src}$ and $\mathbf{a}_{dst}$:

$$e_{ij}^{(h)} = \text{LeakyReLU}\left(\mathbf{a}_{src}^{(h)\top}\mathbf{W}\mathbf{h}_i + \mathbf{a}_{dst}^{(h)\top}\mathbf{W}\mathbf{h}_j\right) \tag{5}$$

This score answers: *"How much should Node i care about Node j's traffic state right now?"*

3) Step 3: Adjacency Masking (GNN vs. Transformer): This is the critical architectural distinction. Attention scores for non-connected intersections are masked to $-\infty$:

$$\alpha_{ij}^{(h)} = \begin{cases} \text{softmax}_j(e_{ij}^{(h)}) & \text{if } A_{ij} > 0 \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

Only physically connected road neighbors can influence each other. A full Transformer omits this step entirely, allowing unconstrained all-to-all attention.

4) Step 4: Head Aggregation: The outputs of all H heads are averaged back into a single d_{out}-dimensional vector per node:

$$\mathbf{h}_i' = \text{ELU}\left(\frac{1}{H}\sum_{h=1}^{H}\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(h)}\mathbf{W}^{(h)}\mathbf{h}_j\right) \tag{7}$$

C. Multi-Hop Stacking with Residual Connections

We stack $K = 3$ GATLayers sequentially (one `ModuleList` of `GATLayer` instances). The first layer maps the 4D input to 128D. After layer 1, each node aggregates information from its direct (1-hop) neighbors. After layer 2, it has indirect awareness of 2-hop neighbors. After layer 3, information has diffused across the 3-hop spatial receptive field.

Crucially, layers 2 and 3 use **residual connections** (additive skip connections):

$$\mathbf{h}^{(k+1)} = \mathbf{h}^{(k)} + \text{GAT}^{(k+1)}(\mathbf{h}^{(k)}, A) \qquad \text{for } k = 1, 2 \tag{8}$$

This prevents gradient degradation in deeper layers and allows the model to preserve local features while enriching them with neighborhood context.

Note: the adjacency matrix A includes weighted self-loops ($A_{ii} = 100$) and edge weights computed as $100/(d_{\text{road}} + 1)$ where d_{road} is the BFS-accumulated road distance between signalized intersections.

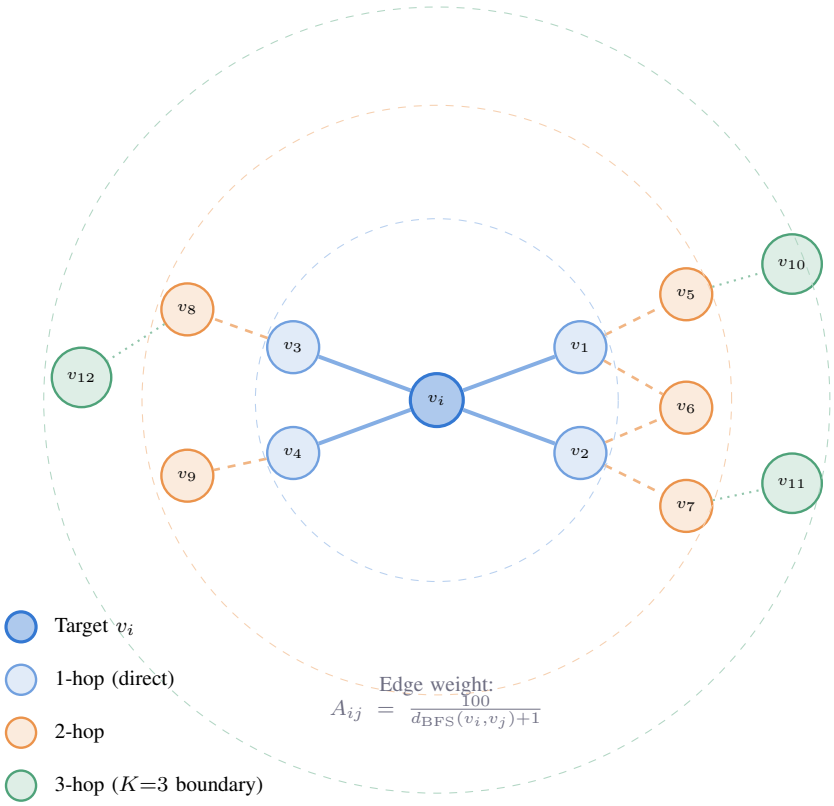

Fig. 2. K-hop spatial receptive field (illustrative; actual Patna STC has N=10). After K=3 GAT layers, a target intersection has aggregated spatial information from all intersections within 3 road-hops. BFS-weighted edges ensure only physically connected nodes exchange information.

D. Temporal Memory via GRU

The spatially enriched features are fed into a standard `nn.GRUCell` (128→128) that maintains a hidden state $\mathbf{h}_{t-1}$ from the previous simulation second. During inference, the spatial feature tensor is flattened to $(B{\cdot}N, 128)$, processed through the GRU, then reshaped back to $(B, N, 128)$:

$$\mathbf{h}_t = \mathrm{GRUCell}(\mathbf{h}^{(K)}_{\text{spatial}}, \mathbf{h}_{t-1}) \tag{9}$$

This gives the model a temporal memory of how traffic *evolves* over time. Importantly, during asynchronous inference, when an intersection is hardware-locked (executing a yellow→green transition), its GRU hidden state is *frozen* (not updated) until the lock expires, preventing temporal corruption.

E. Actor-Critic Heads

The GRU output feeds two separate MLPs:

- **Actor:** Linear$(128 \to 32) \to$ ReLU $\to$ Linear$(32 \to 2) \to$ Softmax — producing per-node action probabilities $\pi(\text{Hold}|s)$ and $\pi(\text{Switch}|s)$.
- **Critic:** Linear$(128 \to 32) \to$ ReLU $\to$ Linear$(32 \to 1)$ — producing per-node state value estimates $V(s)$.

Both heads use orthogonal initialization following the OpenAI PPO standard.

F. PPO Training Objective

The model is trained using the Proximal Policy Optimization (PPO-Clip) algorithm. For each collected trajectory, we compute the Generalized Advantage Estimate (GAE) and optimize the clipped surrogate objective.

Let π_θ denote the current policy and $\pi_{\theta_{\text{old}}}$ the policy used to collect the trajectory. The probability ratio is:

$$\rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \tag{10}$$

The clipped surrogate loss prevents destructively large policy updates:

$$\mathcal{L}^{\text{CLIP}} = -\mathbb{E}_t \left[\min\Big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t \Big) \right] \tag{11}$$

where $\hat{A}_t$ is the advantage estimate and ϵ is the clipping threshold (typically 0.2).

The critic is trained with a standard value function loss:

$$\mathcal{L}^{\text{VF}} = \mathbb{E}_t \left[\left(V_\theta(s_t) - V_t^{\text{target}} \right)^2 \right] \tag{12}$$

An entropy bonus $\mathcal{H}[\pi_\theta]$ is added to encourage exploration. The combined loss is:

$$\mathcal{L} = \mathcal{L}^{\text{CLIP}} + c_1 \mathcal{L}^{\text{VF}} - c_2 \mathcal{H}[\pi_\theta] \tag{13}$$

where c_1 and c_2 are hyperparameter coefficients.

The `evaluate()` function in the model re-runs the full spatial-temporal pipeline (GAT $\rightarrow$ GRU $\rightarrow$ Actor/Critic) on buffered states to compute fresh log-probabilities, value estimates, and entropy for each PPO epoch.

G. Comparative Summary

Table I summarizes the architectural comparison:

TABLE I
ARCHITECTURAL COMPARISON: FULL TRANSFORMER VS. STGAT

Aspect	**Full Transformer**	**Our STGAT**
Attention scope	All-to-all (wasteful)	Road-constrained
Multi-head attention	✓	✓(3 heads)
Topology-aware	×	✓(adj mask)
Temporal memory	Positional enc. (static)	GRU (dynamic recurrent)
Scalability	$O(N^2)$ full	$O(\|E\|)$ sparse
Residual connections	✓	✓(K-hop)
Edge-deployable	× (heavy)	✓(lightweight)

IV. SYSTEM ARCHITECTURE AND PHYSICAL TRANSLATION

The system features a microservices architecture bifurcated into Edge Inference, Cloud Infrastructure, and a live Dashboard UI. Figure 3 illustrates the high-level topology.

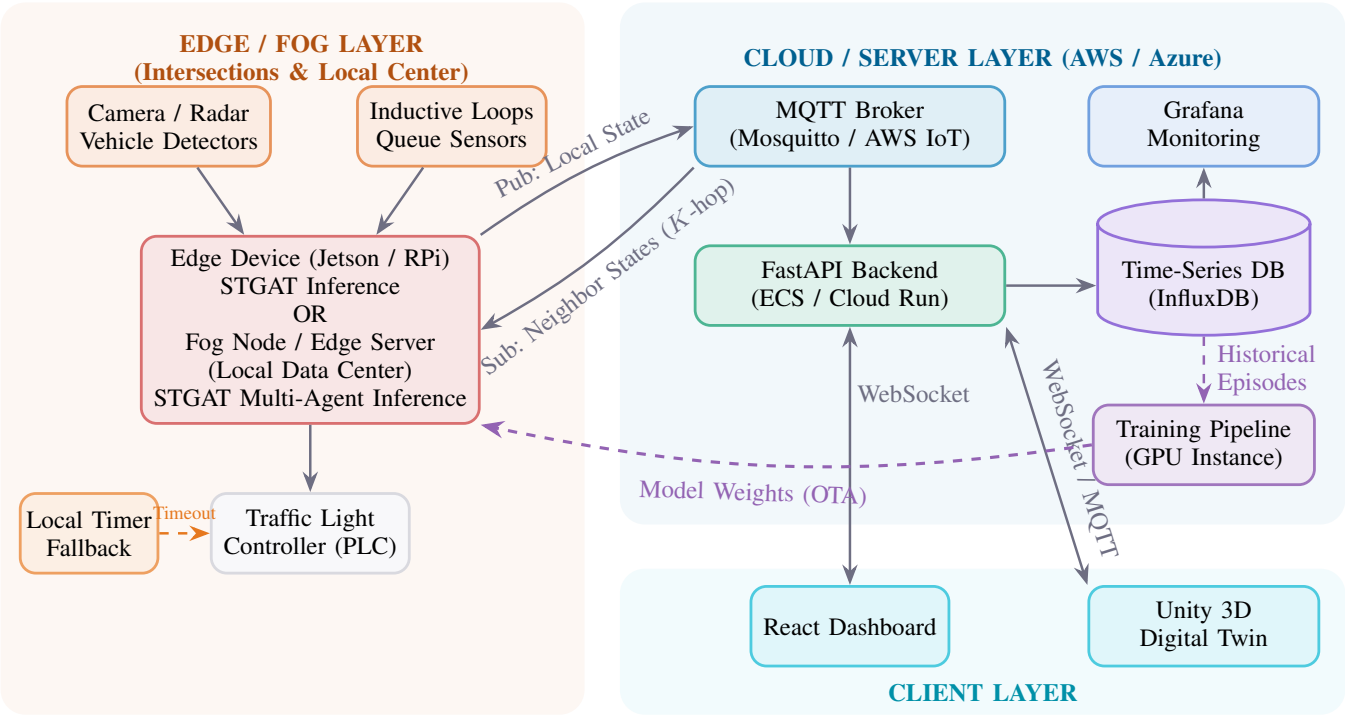

Fig. 3. Proposed IoT Architecture for Real-World Deployment encompassing Edge Sensors, Cloud Aggregation (MQTT/TSDB), GPU Training Pipeline, and Client Dashboard interfaces.

A. *Real-World Implementation and Sensor Mapping*

Translating the SUMO Digital Twin into a physical deployment requires robust edge-sensing arrays.

1) **Cameras and Edge AI/Feature Extraction:** Instead of relying solely on physical inductive loops embedded in asphalt (which degrade and cannot measure queue tail lengths), we propose using pole-mounted CCTV arrays. Edge inference devices (e.g., NVIDIA Jetson Orin Nano) run lightweight YOLOv8 object detection to count vehicles per lane and classify them into PCU types (Car=1.0, Bus=3.0, Tracker=0.5).
2) **Phase State Sensors:** Dedicated PLCs connected to the physical traffic signal cabinet relay the *Current Phase* to the Fog/Edge node.
3) **Edge Computing Pipeline:** The Jetson orchestrates the data, aggregating YOLO detections into a local $[Queue, Wait, Phase]$ tensor. Crucially, it subscribes to the MQTT broker to ingest its K-hop neighbors' tensor streams, constructs the spatial graph, executes a forward pass of the STGAT model, and sends discrete *Hold/Switch* commands back to the PLC.
4) **Fog Computing Pipeline:** Rather than placing an expensive Deep Learning PC at every intersection that relies on localized neighbor broadcasts, a centralized Municipal Edge Server (*Fog Node*) aggregates the $[Queue, Wait, Phase]$ tensors from all intersections in a unified hub. It constructs the holistic spatial graph instantaneously, executes the STGAT forward pass, and dispatches coordinated *Hold/Switch* commands back to the individual intersection PLCs.
5) **Safety Fallback Mechanism:** In cases of network dropout (MQTT failure) where neighbor states are unavailable, the Traffic PLC utilizes a hardware-level timeout to revert to a fixed-cycle signal plan, guaranteeing basic infrastructural safety.

B. *IoT Protocol and Data Backbone*

To simulate this reality, our backend relies on:

- **MQTT Broker (Mosquitto):** Edge metrics are published to `stc/traffic/sensors`.

- **Time-Series DB (InfluxDB):** The telemetry is asynchronously serialized for historical training loops and visualizations.
- **FastAPI WebSockets:** A high-frequency layer pushing the raw tensors directly to the React frontend.

V. Ablation and Comparative Study

To empirically justify the architecture, we evaluated the STGAT approach against several alternatives in our Digital Twin environment.

A. Independent Q-Learning (IQL) vs. GNN

a) Approach: We initially tested Independent Q-Learning, where each intersection hosted its own isolated DQN actor.

b) Drawback: IQL agents rapidly minimized their immediate queues by turning lights green, oblivious to the fact that they were discharging traffic into an already-full downstream adjacent intersection. This caused severe "grid-locking" where lanes became physically blocked, preventing geometric movement.

c) The GNN Advantage: The Graph Attention layer prevented this by allowing intersections to sense downstream blockages up to $K = 3$ hops away, naturally learning to hold red lights if releasing vehicles would induce a gridlock.

B. Graph Diffusion Convolution vs. Graph Attention (STGAT)

a) Approach: Before finalizing the STGAT architecture, we evaluated a Graph Diffusion Convolution approach (often formalized as SA-GNN). In this method, spatial information is aggregated by modeling traffic flow as a diffusion process—specifically, a bidirectional random walk over the adjacency matrix. This allows local states to "diffuse" to K-hop neighbors using a fixed Graph Shift Operator.

b) Drawback: While diffusion convolution successfully extends the receptive field of the agents, it relies on static transition matrices derived purely from the physical road topology (e.g., node degrees). Consequently, it aggregates spatial information uniformly or based on fixed structural weights, remaining blind to the real-time, asymmetric severity of approaching traffic.

c) The STGAT Advantage: The Multi-Head Graph Attention Network (STGAT) fundamentally overcomes this limitation. Instead of passively diffusing states, STGAT computes dynamic attention coefficients between intersections based on their live feature embeddings. This allows an agent to proactively shift its "attention" to heavily weight a critical upstream intersection experiencing a sudden traffic surge, while safely ignoring an adjacent intersection with empty lanes.

Mathematically, the diffusion convolution at hop k computes:

$$\mathbf{Y}^{(k)} = (\mathbf{T}_f^k + \mathbf{T}_r^k)\mathbf{X} \qquad \text{where } \mathbf{T}_f = A^\top D_{\text{out}}^{-1},\ \mathbf{T}_r = AD_{\text{in}}^{-1} \tag{14}$$

The final aggregation uses *learnable* softmax-normalized hop weights $\{w_k\}$:

$$\mathbf{Z} = \sum_{k=0}^{K-1} \frac{e^{w_k}}{\sum_j e^{w_j}} \cdot \mathbf{Y}^{(k)} \tag{15}$$

While w_k are learnable, the transition matrices $\mathbf{T}_f, \mathbf{T}_r$ remain *fixed* throughout training—derived solely from static graph topology. In contrast, STGAT's attention coefficients $\alpha_{ij}^{(h)}$ are recomputed at *every time step* based on live node features, enabling truly demand-responsive spatial reasoning.

C. Baseline Fixed-Timing

a) Approach: Traditional Cyclic Signal operation.

b) Findings: Under heavy load, the baseline's average maximum wait time exploded linearly, while STGAT behaved as a dynamic pressure-release valve, flushing queues the moment exponential wait penalties accrued.

VI. Challenges Faced and Engineering Solutions

A. Synchronous Time-Jumping vs. Asynchronous Hardware Locks

a) Challenge: Standard RL environments execute a global `env.step()` that fast-forwards the entire simulation to the end of a phase transition (e.g., 13 seconds). In a real-world network, freezing the entire city while one intersection changes its lights destroys global traffic flow and prevents independent operation.

b) Proposed idea: We bypassed the global step function and implemented a 1 Hz real-time asynchronous loop. A custom *Action Masking* wrapper was engineered using independent hardware lock arrays. If an intersection is executing a phase transition, the Python wrapper safely masks the AI's commands and freezes its specific GRU hidden state (`h_prev`), allowing other intersections to operate and transition completely independently.

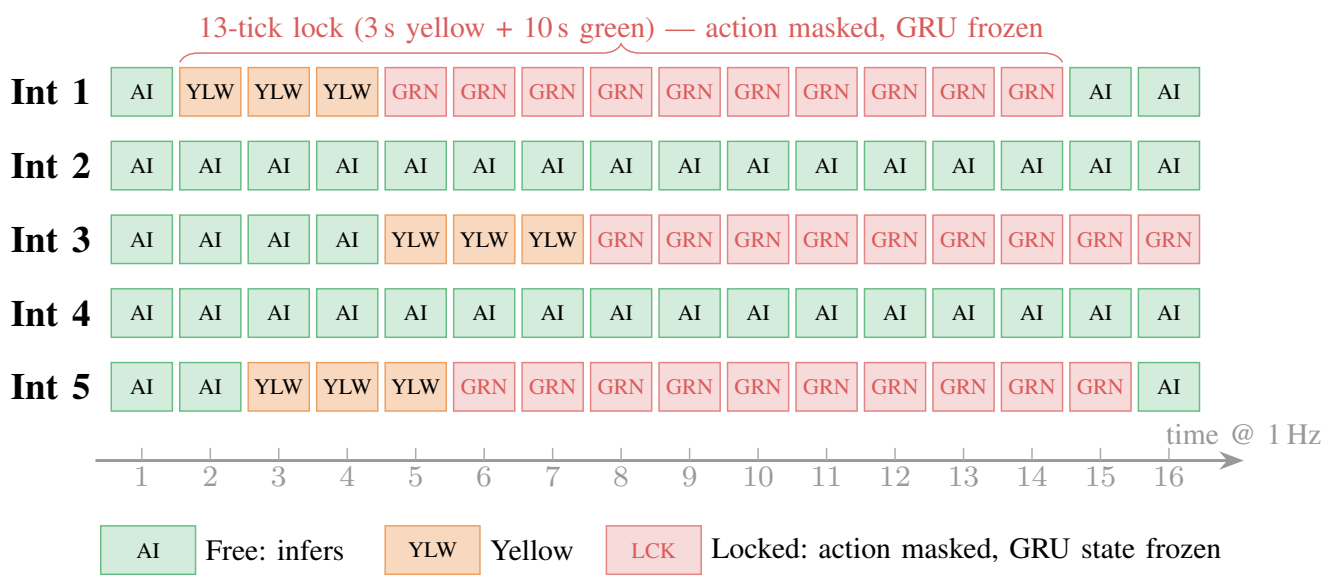

Fig. 4. Asynchronous 1 Hz inference timeline. Each intersection maintains an independent hardware lock counter (13 ticks = 3s yellow + 10s green). Locked intersections have their AI actions masked and GRU states frozen.

B. Out-of-Distribution "Gridlock Panic" under Asymmetric Flow

a) Challenge: During unidirectional stress tests (e.g., 2,500 vehicles traveling strictly West-to-East), the STGAT agent suffered from Out-of-Distribution (OOD) hallucinations. Because it was trained heavily in symmetric gridlock, the presence of a massive queue triggered a panic, causing the agent to rapidly cycle phases despite the cross-streets being completely empty, effectively halting the major arterial flow.

b) Idea: We introduced a hybrid AI-Hardware *Demand-Responsive Action Mask*. Before the edge device transmits a phase-switch command to the PLC, it queries the red-light approach lanes. If the cross-traffic queue is exactly zero, the hardware vetoes the AI and forces the main street to Rest-in-Green.

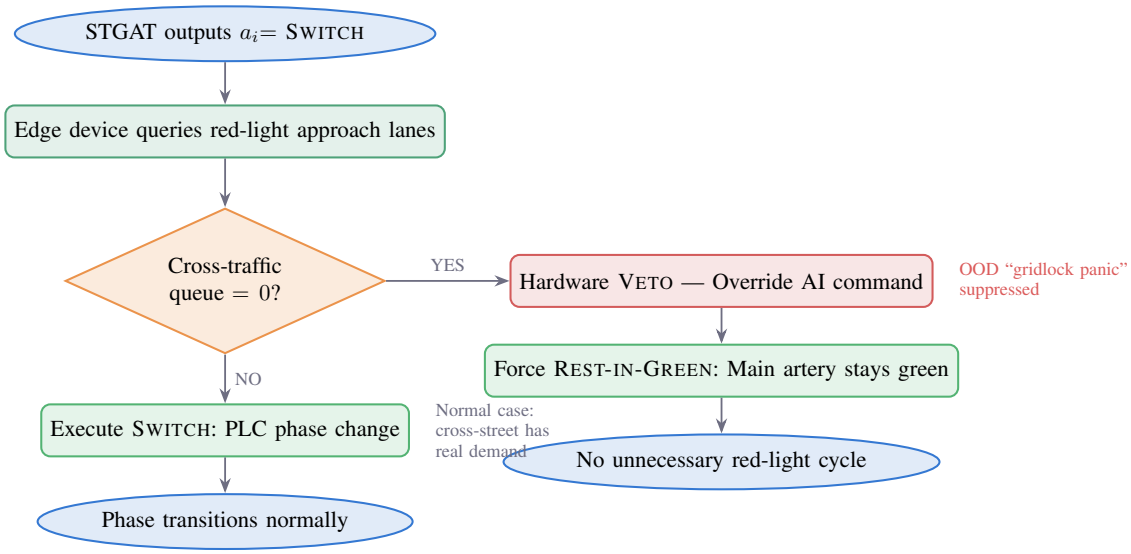

Fig. 5. Demand-responsive action mask flow. Before executing a phase switch command, the edge device queries red-light approach lanes for halting vehicles. If cross-traffic queue = 0, the switch is vetoed and the main artery remains green.

C. Topological Blindness in Graph Construction

a) Challenge: Real-world SUMO networks (like Patna STC) map single physical roads using dozens of microscopic geometry edges and unsignalized nodes (e.g., yield signs, bends). Initial static Adjacency Matrix formulations only checked one edge deep, failing to realize that two signalized intersections were connected, leaving the GAT layer effectively blind.

b) Proposed idea: The environment's matrix generation was rewritten utilizing a Breadth-First Search (BFS) algorithm. The BFS traverses the physical network, aggregating distances and bypassing unsignalized "dummy" nodes until it strikes the next logical traffic signal, seamlessly collapsing physical geometry into a highly accurate Homogeneous Logical Graph for the attention layer.

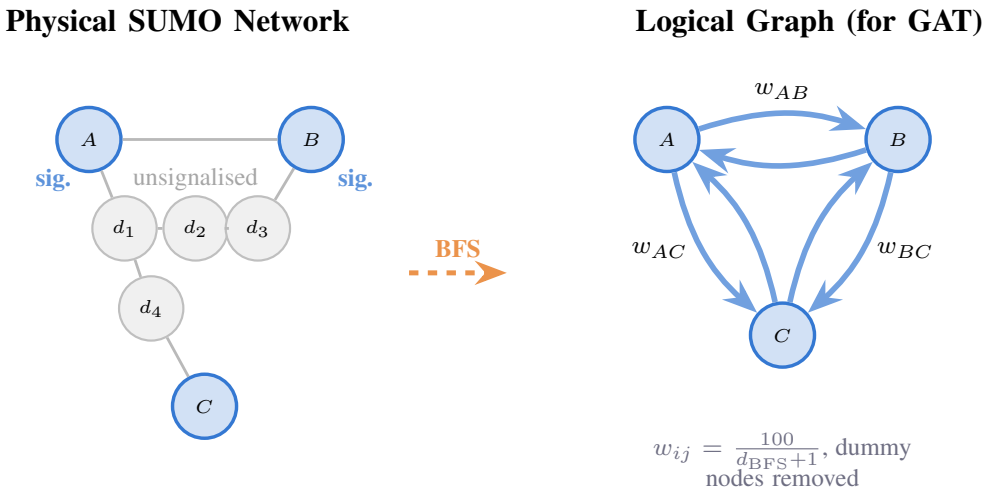

Fig. 6. BFS-based logical graph construction. Unsignalised dummy nodes (yield signs, geometry bends) are skipped; road distances are accumulated to yield a compact homogeneous graph over the N=10 signalised intersections.

D. Fair Comparative Benchmarking

a) Challenge: Initial comparison charts between the Baseline and Agent showed immense, inexplicable variance. Investigation revealed differing mathematical standards: the RL agent used scaled tensor arrays, while the baseline read raw TraCI integer metrics. Moreover, differing SUMO random seeds generated completely separate traffic scenarios.

b) Proposed idea: We enforced a strict deterministic seed passed directly from the React UI to both Python runners. All normalizations were unified, ensuring the Baseline runner evaluates identical demand traffic geometries.

E. WebSocket ASGI Concurrency

a) Challenge: As episodes began, the React UI re-renders repeatedly severed the WebSocket connection, causing fatal `Unexpected ASGI message` runtime exceptions on the backend when it attempted to stream step-data.

b) Proposed idea: State management was decoupled using static `useRef` hooks to track `run_ids` without destroying and rebuilding the websocket tree mid-cycle.

VII. Hardware and Software Specifications

A. Physical Deployment Stack

- **Visual Sensing:** 1080p IP Cameras per approach.
- **Edge Compute:** NVIDIA Jetson Xavier NX / Orin Nano (15W - 25W profile).
- **PLC Interface:** NTCIP compliant traffic controller hardware.
- **Telemetry:** 4G/5G cellular modems running MQTT over TLS 1.2.

B. Simulation/Training Stack

- **Simulator:** Eclipse SUMO $\geq$ 1.18.0. Training uses `libsumo` (in-process, faster), while inference uses `traci` (subprocess mode, allowing SUMO-GUI Digital Twin).
- **Training Backend:** Python 3.10+, PyTorch 2.0+ (CUDA), FastAPI, Uvicorn (ASGI).
- **Training Hardware:** Dedicated NVIDIA RTX 3060 12GB minimum.
- **IoT Stack:** Eclipse Mosquitto 2.0+ (MQTT), InfluxDB 2.x (TSDB), Docker Compose.
- **Frontend & Rendering:** Node.js 18+, React (Vite.js) for 2D metrics, and Unity 3D for live spatial rendering.

VIII. Conclusion

The fusion of Graph Attention Networks with Proximal Policy Optimization directly addresses the failures of isolated and pre-timed traffic control paradigms. By introducing $K{=}3$ hop spatial awareness across 10 signalized intersections, the STGAT model successfully prevents ripple-effect gridlocks that plague both Independent Q-Learning and fixed-timing baselines. The demand-responsive action mask further stabilizes the agent against out-of-distribution traffic patterns.

The engineering contributions are equally significant: asynchronous 1 Hz hardware locks decouple intersection control from global simulation stepping, BFS-based adjacency construction handles complex real-world SUMO topologies, and deterministic seeding ensures fair benchmarking against baseline controllers.

Paired with a robust microservice IoT architecture—featuring MQTT telemetry, InfluxDB time-series persistence, WebSocket streaming, and a real-time React monitoring dashboard—this work provides a transparent, scalable bridge between academic Deep Reinforcement Learning models and production-ready Smart City deployments.

IX. Future Work

Several directions remain for extending this architecture:

A. Model Quantization for Edge Deployment

The current STGAT model operates in FP32 precision. For deployment on power-constrained edge devices (Jetson Orin Nano, 15W), the model should be quantized to INT8 using NVIDIA TensorRT or PyTorch's built-in quantization toolkit. This would reduce inference latency from $\sim$8ms to $<$1ms and cut memory footprint by $4\times$, enabling multi-intersection inference on a single edge node.

B. Camera-Based Perception Integration

The current system assumes TraCI provides perfect vehicle state information. In physical deployment, perception noise from camera-based detection (YOLOv8/v9) will introduce uncertainty in PCU counts and queue measurements. Future work should:

- Train the STGAT agent with *observation noise injection* (Gaussian noise on q_i and w_i) during simulation to build robustness.
- Implement a Kalman filter or Bayesian fusion layer between the YOLO detector output and the GNN input tensor.

C. Multi-Objective Reward Shaping

The current reward $r_i = -(q_i + 0.5w_i) - 15 \cdot \mathbb{1}[\text{switch}]$ treats all vehicle types equally. Future formulations could incorporate:

$$r_i^{(t)} = -\left(q_i + \omega w_i + \beta \cdot \text{CO}_2(i,t) + \gamma \cdot d_{\text{emergency}}(i,t)\right) \tag{16}$$

where CO_2 penalizes idle emissions (proportional to PCU-weighted queue $\times$ time) and $d_{\text{emergency}}$ provides priority preemption for ambulances/fire trucks detected via V2X or YOLO classification.

D. Federated Learning across Cities

The current model is trained centrally on one city topology. Federated Learning could enable multiple cities to collaboratively improve a shared STGAT backbone without exchanging raw traffic data—only gradient updates—addressing both data privacy and domain generalization.

E. Cloud-Native Scaling

Transitioning from local Docker Compose to managed cloud services:

- **MQTT:** AWS IoT Core or Azure IoT Hub for managed broker with TLS mutual authentication.
- **TSDB:** Amazon Timestream or InfluxDB Cloud for auto-scaling time-series storage.
- **Inference:** AWS Lambda@Edge or Azure IoT Edge for serverless model serving.

References

[1] Krajzewicz, D., Erdmann, J., Behrisch, M., and Bieker, L., "Recent Development and Applications of SUMO - Simulation of Urban MObility," *Int'l Journal On Advances in Systems and Measurements*, vol. 5, no. 3&4, pp. 128–138, 2012.

[2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O., "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[3] Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., and Bengio, Y., "Graph Attention Networks," *ICLR*, 2018.

[4] Zhang, Y., Yu, Z., Zhang, J., Wang, L., Luan, T. H., Guo, B., and Yuen, C., "Learning Decentralized Traffic Signal Controllers with Multi-Agent Graph Reinforcement Learning," *arXiv preprint arXiv:2311.03756*, 2023.

Appendix A
Live Dashboards and Digital Twin

The theoretical architectures proposed in this paper have been fully translated into the live monitoring interfaces demonstrated below.

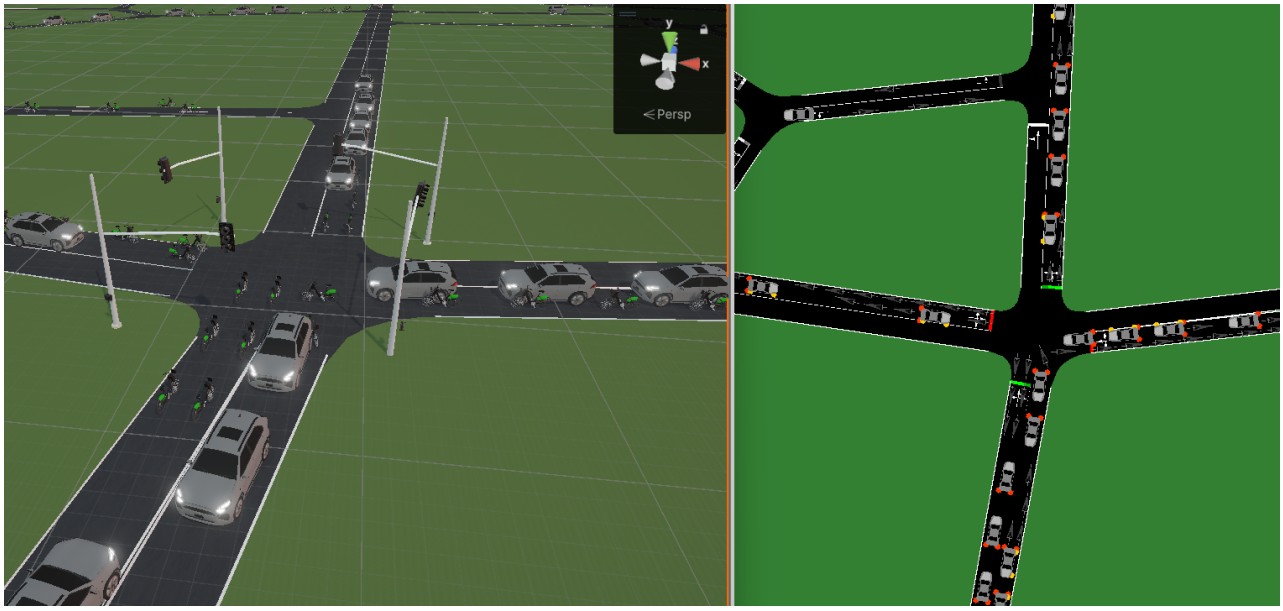

Fig. 7. The Unity 3D Digital Twin rendering the physical Patna STC environment, ingesting real-time metrics and phase states from the cloud backend.

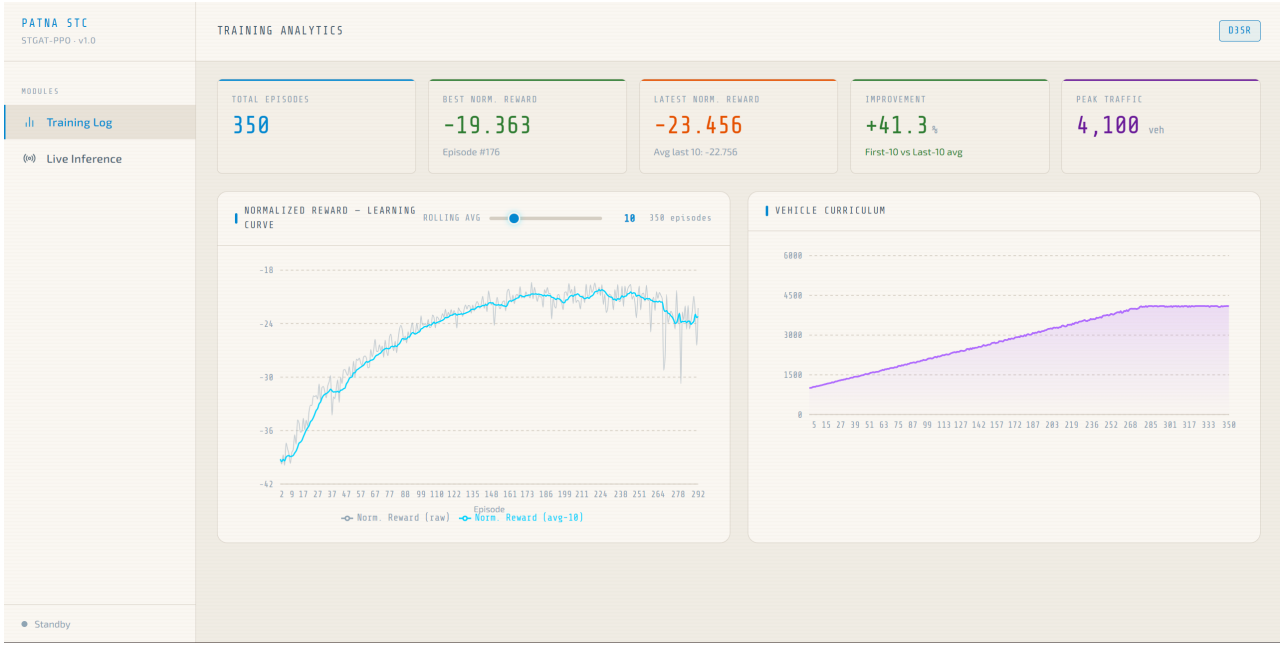

Fig. 8. Last training analytics

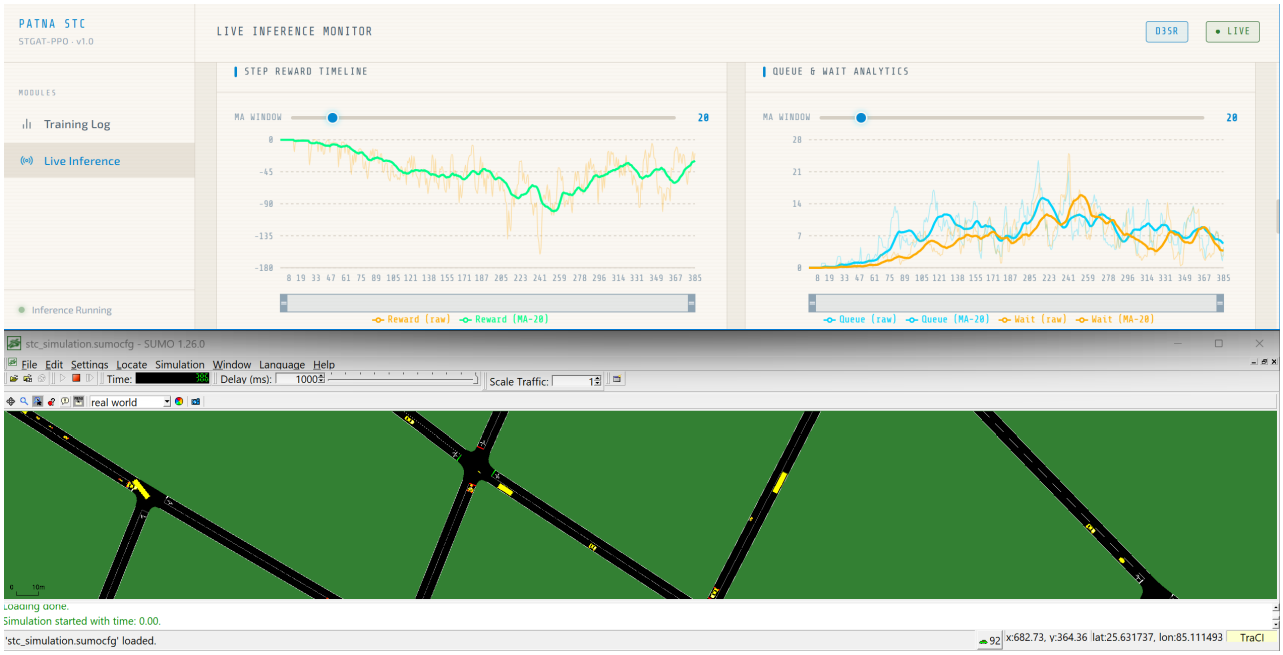

Fig. 9. Live inference and simulation analytics

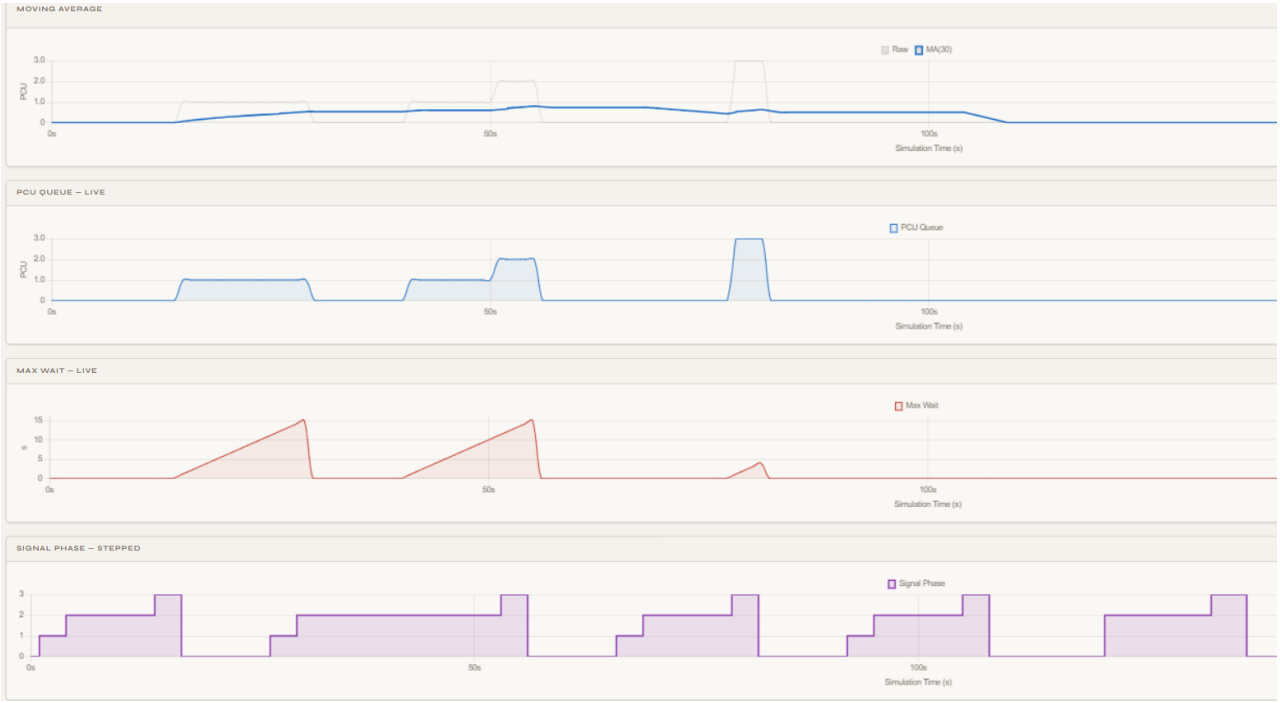

Fig. 10. Single intersection analytics

Appendix B
Author Contributions

This project was conceptualized, engineered, and evaluated collaboratively by Team D3SR (IIT ISM Dhanbad). Primary individual contributions are outlined as follows:

a) ***Divya Prakash Sinha****:* Developed the procedural 3D road mesh network generation in Unity by parsing traffic topology data. Implemented an MQTT-over-WebSocket client to visualize real-time traffic conditions and signal states across junctions in 3D.

b) ***Dyuti Ballav Paul****:* Engineered the STGAT reinforcement learning model. Formulated the GNN spatial embedding pipeline, multi-head attention logic, and RL. Conducted the baseline algorithm ablation mathematics and reward function engineering.

c) ***Devansh Gupta****:* Engineered the Cloud Backend API infrastructure. Developed the FastAPI WebSocket asynchronous data-streaming pipelines and integrated the InfluxDB time-series database for historical RL telemetry persistence.

d) ***Shreyas Kumar Patel****:* Designed and implemented a React/Vite dashboard integration, enabling real-time visualization of traffic metrics and system performance. Developed WebSocket-based data ingestion and transformation pipelines to handle high-frequency MQTT streams efficiently.

e) ***Rishit Das****:* Edge and IoT Protocols engineer. Orchestrated the Mosquitto MQTT message brokering between the inference endpoints and backend, and authored the physical deployment topology for NVIDIA Jetson nodes and automated camera integration.